

How many attention heads do you need to do XOR?

Originally published on [LessWrong](#)

Karthik Viswanathan

2 April 2026

*You need **at least two attention heads to do XOR***, and we will find that it is a surprisingly crisp result which uses only a few lines of algebra.

XOR is one of the most widely used Boolean functions that is not linearly separable, making it a [classic test](#) of computational expressivity, from perceptrons to modern transformer circuits. Understanding how many attention heads are needed to compute it tells us something fundamental about what individual components of a transformer can and cannot represent.

Introduction

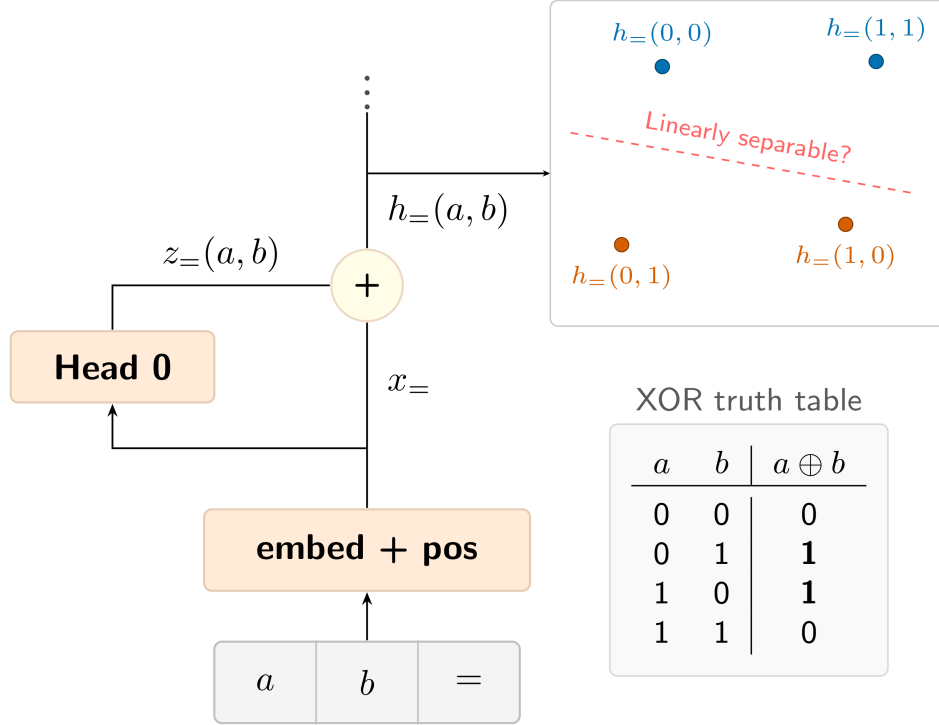


Figure 1: **Computing XOR using an attention head.** Two input bits (a, b) and the query token $=$ are embedded and processed by a single attention head with a skip connection. Is the residual stream of the query token $h_=(a, b)$ linearly separable into XOR classes?

What Boolean functions can a linear probe recover from the residual stream after the first attention update? This isolates the expressivity of attention itself since downstream MLPs or layer norms could solve XOR on their own, but we want to know what is already linearly accessible before any further processing. It turns out that **a single attention head** already makes **OR** and **AND** linearly separable, but it **cannot do XOR**¹. We also give an explicit construction showing that **two attention heads suffice**.

We will show this using the setup outlined in the above figure, checking if the residual stream of the query token ($h_=(a, b)$) is linearly separable into XOR classes. Since the skip connection $x_=($ is a constant offset across all inputs, linear separability depends only on the attention update $z_=(a, b)$, which we analyze for the rest of the post.

$$\text{OR}(a, b) = \max(a, b)$$

$$^1\text{AND}(a, b) = \min(a, b) \quad \text{XOR is denoted by } \oplus \text{ in equations.}$$

$$\text{XOR}(a, b) = (a + b) \bmod 2$$

A short sketch of the proof

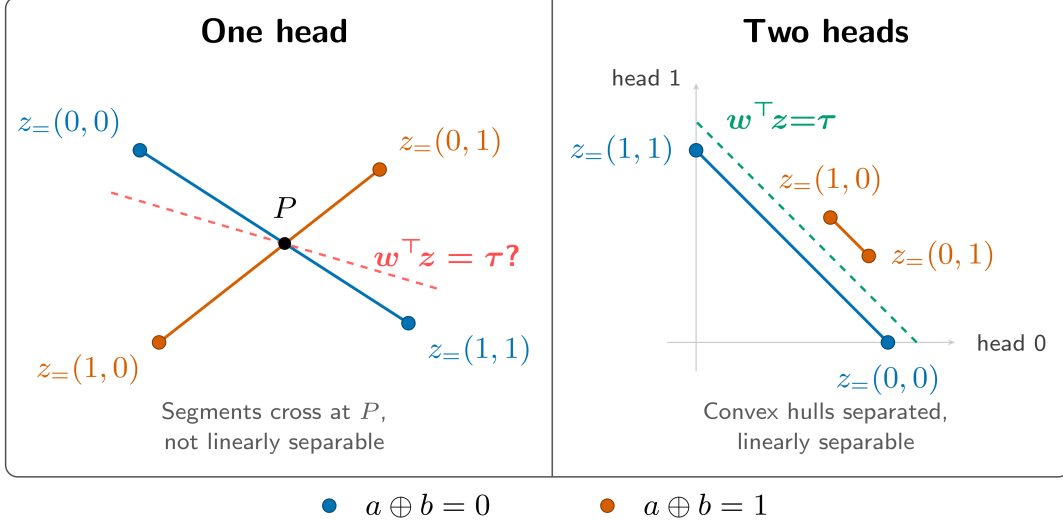


Figure 2: **One attention head cannot compute XOR, but two can.** With a single head, the line segments connecting same-class points always intersect, ruling out linear separability. With two heads, each head’s segments still intersect individually, but linear separability emerges in the sum of their contributions.

In the single-head case, we will show that the attention update results in a point P that lies in the intersection of the line segments connecting same-class points², i.e.,

$$P \in [z=(0,0), z=(1,1)] \cap [z=(0,1), z=(1,0)].$$

This rules out linear separability: any separating hyperplane must place each class’ segment entirely on one side, but two segments that intersect cannot be separated by a hyperplane.

However, two heads are enough. Conceptually, one head detects 0 and the other detects 1. On the mixed inputs 01 and 10, **both** heads contribute; on 00 and 11, only one of them does. A linear readout can then separate the mixed cases from the same-bit ones.

This means you need at least two attention heads to do XOR.

Corollary: Parity detection in modular addition tasks. For any two distinct token values $A \neq B \in \mathbb{Z}_p$, the first attention update cannot linearly separate the “same” inputs $\{(A, A), (B, B)\}$ from the “mixed” inputs $\{(A, B), (B, A)\}$, since restricting to these four inputs recovers exactly the XOR structure. Note that this is a statement about each 2-element restriction separately, not about full parity over all p^2 inputs.

²We call $z=(0,0)$ and $z=(1,1)$ the XOR-negative outputs (where $a \oplus b = 0$) and $z=(0,1)$ and $z=(1,0)$ the XOR-positive outputs (where $a \oplus b = 1$).

Setup

We work with sequences of length 3 over the vocabulary $\{0, 1, =\}$. On input $(a, b) \in \{0, 1\}^2$, the model sees the sequence $(a, b, =)$.

Self Attention 101

Each token $t \in \{0, 1, =\}$ has a token embedding $e_t \in \mathbb{R}^d$, and each position $j \in \{a, b, =\}$ has a positional embedding $\text{pos}_j \in \mathbb{R}^d$. The embedded sequence is

$$x_a = e_a + \text{pos}_a, \quad x_b = e_b + \text{pos}_b, \quad x_ = e_ + \text{pos}_=.$$

A single attention head is parameterized by the query, key and value matrices denoted as $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$ respectively. The = token attends to all three positions via softmax attention, resulting in the residual stream $h_=(a, b)$:

$$h_=(a, b) = x_ + \sum_{j=1}^3 \alpha_j(a, b) W_V x_j$$

where $\alpha_j(a, b)$ is the attention weight from the j^{th} key to the = token given as:

$$\alpha_j(a, b) = \frac{\exp(x_^\top W_Q^\top W_K x_j)}{\sum_{k=1}^3 \exp(x_^\top W_Q^\top W_K x_k)}$$

The = token's residual stream after the attention head has two parts: the **skip connection** $x_ =$ (its original embedding) and the **attention update** $z_=(a, b)$ (the new information it gathered by attending to a and b):

$$h_=(a, b) = x_ + z_=(a, b).$$

Since $x_ =$ doesn't depend on the input bits at all, any probe can fold it into its threshold τ . So the only thing that matters for classification is the attention update $z_=(a, b)$.

Let $v_j := W_V x_j$ denote the value vector at position j . The attention update is then a convex combination of these value vectors, weighted by the attention probabilities $\{p_a, p_b, p_=\}$:

$$z_=(a, b) := p_a v_a + p_b v_b + p_ = v_ =, \quad p_a + p_b + p_ = = 1, \quad p_j > 0.$$

The attention probabilities are the softmax of the raw attention logits $\sigma_a, \sigma_b, \sigma_ =$, which measure how strongly the = token's query matches each key:

$$\sigma_a := \exp(x_^\top W_Q^\top W_K x_a), \quad \sigma_b := \exp(x_^\top W_Q^\top W_K x_b), \quad \sigma_ = := \exp(x_^\top W_Q^\top W_K x_ =)$$

resulting in the following attention weights:

$$p_j = \frac{\sigma_j}{\sigma_a + \sigma_b + \sigma_{=}}.$$

So we can equivalently write the attention update directly in terms of the σ 's:

$$z_{=}(a, b) = \frac{\sigma_a v_a + \sigma_b v_b + \sigma_{=} v_{=}}{\sigma_a + \sigma_b + \sigma_{=}}.$$

We now ask: can a hyperplane separate the four attention outputs $\{z_{=}(a, b)\}_{a, b \in \{0, 1\}}$ into the XOR classes?

$$w^\top z_{=}(a, b) > \tau \iff a \oplus b = 1.$$

One attention head can do OR and AND³

It is worth noticing that XOR is the *first* interesting example here since a single head can already do **OR** and **AND**.

Here is a simple way to see it. Take a head that attends to 1 tokens and writes a positive value when it reads one.⁴ Its output increases with the number of ones, so it produces a score that is monotone in $a + b$:

$$a + b = 0 \mapsto \text{low}, \quad a + b = 1 \mapsto \text{medium}, \quad a + b = 2 \mapsto \text{high}.$$

Now thresholding does the rest:

- threshold below the medium value gives $a + b \geq 1$, i.e. **OR**;
- threshold between medium and high gives $a + b = 2$, i.e. **AND**.

So one head can do monotone threshold functions of $a + b$ just fine. XOR is different because it is not monotone: it fires at $a + b = 1$ but not at $a + b = 0$ or $a + b = 2$. No single threshold can pick out the middle value from both sides.

One attention head cannot do XOR

We now give a short proof that a single attention head cannot do XOR.

The key identities

In the Setup section, we saw that the attention update $z_{=}(a, b)$ takes the form

$$z_{=}(a, b) = \frac{\sigma_a v_a + \sigma_b v_b + \sigma_{=} v_{=}}{\sigma_a + \sigma_b + \sigma_{=}} := \frac{N(a, b)}{D(a, b)}.$$

where $N(a, b) := \sigma_a v_a + \sigma_b v_b + \sigma_{=} v_{=}$ and $D(a, b) := \sigma_a + \sigma_b + \sigma_{=}$. Note that $D(a, b) > 0$, a fact we will use shortly.

³To clarify, it performs these operations independently (either OR or AND), not simultaneously

⁴For example, this can be done by using a value matrix that is “parallel” to $e(1)$ and “orthogonal” to $e(0)$.

The key structural fact is that $N(a, b)$ and $D(a, b)$ each split into an a -only term, a b -only term, and a constant:

$$N(a, b) = \underbrace{\sigma_a v_a}_{a\text{-only}} + \underbrace{\sigma_b v_b}_{b\text{-only}} + \underbrace{\sigma_{=v=}}_{\text{const}}, \quad D(a, b) = \underbrace{\sigma_a}_{a\text{-only}} + \underbrace{\sigma_b}_{b\text{-only}} + \underbrace{\sigma_{=}}_{\text{const}}.$$

Because of this, summing over the main diagonal $\{(0, 0), (1, 1)\}$ versus the off-diagonal $\{(0, 1), (1, 0)\}$ yields identical totals — in both cases you collect exactly one copy each of the $a=0$ and $a=1$ contributions, and one copy each of the $b=0$ and $b=1$ contributions. This gives the **key identities**:

$$N(0, 0) + N(1, 1) = N(0, 1) + N(1, 0) = \mathcal{N},$$

$$D(0, 0) + D(1, 1) = D(0, 1) + D(1, 0) = \mathcal{D}.$$

Line segments connecting the same class intersect

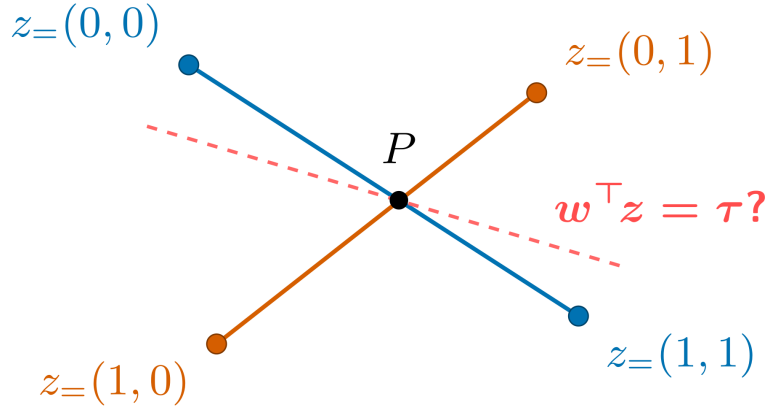


Figure 3: **The geometric obstruction.** The segment connecting the XOR-negative outputs $[z=(0, 0), z=(1, 1)]$ (blue) always crosses the segment connecting the XOR-positive outputs $[z=(0, 1), z=(1, 0)]$ (orange) at a point P . Any hyperplane $w^\top z = \tau$ must place P on both sides simultaneously, making linear separation impossible.

We now show that the positive-class segment always intersects the negative-class segment, ruling out linear separability.

Recall from the definition $z_=(a, b)$, we get $N(a, b) = D(a, b) z_=(a, b)$. Substituting this into the diagonal identity $\mathcal{N} = N(0, 0) + N(1, 1) = N(0, 1) + N(1, 0)$ gives

$$\begin{aligned} \mathcal{N} &= D(0, 0) z_=(0, 0) + D(1, 1) z_=(1, 1) \\ &= D(0, 1) z_=(0, 1) + D(1, 0) z_=(1, 0). \end{aligned}$$

Dividing both expressions by $\mathcal{D} = D(0, 0) + D(1, 1) = D(0, 1) + D(1, 0)$, we obtain

$$\begin{aligned} P &:= \frac{\mathcal{N}}{\mathcal{D}} \\ &= \frac{D(0, 0) z_{\leq}(0, 0) + D(1, 1) z_{\leq}(1, 1)}{D(0, 0) + D(1, 1)} \\ &= \frac{D(0, 1) z_{\leq}(0, 1) + D(1, 0) z_{\leq}(1, 0)}{D(0, 1) + D(1, 0)}. \end{aligned}$$

Since every $D(a, b) > 0$, both sides are **convex combinations**: the left side is a point on the segment $[z_{\leq}(0, 0), z_{\leq}(1, 1)]$ and the right side is a point on $[z_{\leq}(0, 1), z_{\leq}(1, 0)]$. So point P satisfies

$$P \in [z_{\leq}(0, 0), z_{\leq}(1, 1)] \cap [z_{\leq}(0, 1), z_{\leq}(1, 0)].$$

In words: the segment joining the two XOR-negative hidden states always crosses the segment joining the two XOR-positive hidden states.

This immediately rules out linear separability. If a probe $L(z) = w^\top z - \tau$ had $L < 0$ at both $z_{\leq}(0, 0)$ and $z_{\leq}(1, 1)$, then by convexity $L < 0$ on the entire segment $[z_{\leq}(0, 0), z_{\leq}(1, 1)]$. Likewise $L > 0$ on $[z_{\leq}(0, 1), z_{\leq}(1, 0)]$. But P lies on both segments, forcing $L(P) < 0$ and $L(P) > 0$ simultaneously, resulting in a contradiction. $\square$

Conclusion. A single attention head with a linear readout cannot compute XOR.

Two heads can do XOR

Now we switch from one head to **two parallel heads**. For the existence proof, it is enough to write the residual update as the sum of the two head outputs:

$$z_{\leq}(a, b) = y_{\leq}^{(0)}(a, b) + y_{\leq}^{(1)}(a, b),$$

where

$$y_{\leq}^{(r)}(a, b) := \sum_{j=1}^3 \alpha_j^{(r)}(a, b) W_V^{(r)} x_j.$$

The idea is simple:

- Head 0 softly detects token 0 and writes in one direction;
- Head 1 softly detects token 1 and writes in an orthogonal direction.

Then the mixed inputs 01 and 10 are exactly the cases where **both** heads contribute. This is illustrated in the right panel of the figure in the introduction. Although each head's class segments still intersect individually, their combined outputs live in a 2D subspace where $[z_{\leq}(0, 0), z_{\leq}(1, 1)]$ and $[z_{\leq}(0, 1), z_{\leq}(1, 0)]$ pull apart, and a separating hyperplane $w^\top z = \tau$ fits between them.

An explicit construction

Let's work in $d = 3$, with no positional embeddings, and choose the token embeddings as follows:

$$e_0 = (1, 0, 0), \quad e_1 = (0, 1, 0), \quad e_{=} = (0, 0, 1).$$

We choose the query, key, value and output matrices for each head as shown below, along with the resulting attention scores and weights from the $=$ query position.

	Head 0	Head 1
W_Q	I	I
W_K	$e_{=}e_0^\top$	$e_{=}e_1^\top$
W_V	$e_0e_0^\top$	$e_1e_1^\top$
W_O	I	I

The construction is symmetric by design:

- Head 0 attends preferentially to token 0 and writes in the e_0 direction
- Head 1 attends preferentially to token 1 and writes in the e_1 direction

Since each raw attention logit is either 0 or 1, the softmax exponentiates to either $1 = \exp(0)$ or $e = \exp(1)$, which is why e appears throughout the tables below. Each entry is the triplet of softmax attention weights $(\alpha_a, \alpha_b, \alpha_{=})$ that the $=$ query assigns to positions $(x_a, x_b, x_{=})$ respectively, for that head and input. The weights are non-negative and sum to 1.

Input	Head 0	Head 1
(0, 0)	$\left(\frac{e}{2e+1}, \frac{e}{2e+1}, \frac{1}{2e+1}\right)$	$\left(\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\right)$
(0, 1)	$\left(\frac{e}{e+2}, \frac{1}{e+2}, \frac{1}{e+2}\right)$	$\left(\frac{1}{e+2}, \frac{e}{e+2}, \frac{1}{e+2}\right)$
(1, 0)	$\left(\frac{1}{e+2}, \frac{e}{e+2}, \frac{1}{e+2}\right)$	$\left(\frac{e}{e+2}, \frac{1}{e+2}, \frac{1}{e+2}\right)$
(1, 1)	$\left(\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\right)$	$\left(\frac{e}{2e+1}, \frac{e}{2e+1}, \frac{1}{2e+1}\right)$

We can now compute the attention update from each head across all four inputs.

Input	Head 0	Head 1	Head 0 + Head 1
(0, 0)	$\frac{2e}{2e+1} e_0$	0	$\frac{2e}{2e+1} e_0$
(0, 1)	$\frac{e}{e+2} e_0$	$\frac{e}{e+2} e_1$	$\frac{e}{e+2} (e_0 + e_1)$
(1, 0)	$\frac{e}{e+2} e_0$	$\frac{e}{e+2} e_1$	$\frac{e}{e+2} (e_0 + e_1)$
(1, 1)	0	$\frac{2e}{2e+1} e_1$	$\frac{2e}{2e+1} e_1$

The **same-bit inputs** (0, 0) and (1, 1) each **activate only one head**, while the mixed inputs (0, 1) and (1, 0) activate both heads equally. The mixed inputs therefore have a strictly larger total activation, which a linear readout can exploit.

Choosing $w = e_0 + e_1$, the probe score $w^\top z_=(a, b)$ takes only two distinct values:

$$w^\top z_=(0, 0) = w^\top z_=(1, 1) = \frac{2e}{2e+1} \approx 0.84, \quad w^\top z_=(0, 1) = w^\top z_=(1, 0) = \frac{2e}{e+2} \approx 1.15.$$

The mixed inputs score strictly higher, so any threshold $\tau \in (0.84, 1.15)$ together with $w = e_0 + e_1$ correctly classifies XOR.

Conclusion. Two attention heads are sufficient to compute XOR with a linear readout.

Takeaway

The single-head impossibility holds for any embedding dimension, any positional encoding, and any attention parameters. It is a purely structural consequence of how softmax attention computes a weighted average: the additive decomposition of the numerator and denominator forces the class segments to cross, ruling out any function that requires separating the diagonal $\{(0, 0), (1, 1)\}$ from the off-diagonal $\{(0, 1), (1, 0)\}$.

Two heads break this by giving the outputs a second dimension to spread into. Each head’s outputs still satisfy the crossing constraint individually, but the sum of two heads’ contributions lives in a 2D subspace where the class segments pull apart. The mixed inputs $(0, 1)$ and $(1, 0)$ are the only cases where both heads contribute, creating a gap that a linear readout can exploit.

More broadly, the segment-crossing argument applies whenever a single attention head must separate the diagonal $\{(0, 0), (1, 1)\}$ from the off-diagonal $\{(0, 1), (1, 0)\}$. XOR is the simplest such function, but the same geometric obstruction rules out any target that requires this checkerboard sign pattern. This geometric obstruction is reminiscent of the [topological constraints on neural network classification](#) analysed by Chris Olah in his blogpost: just as low-dimensional networks cannot separate linked manifolds without sufficient width, a single attention head cannot separate the XOR classes because its outputs are forced into a configuration where the class segments cross.

In a single-layer, attention-only model with a linear readout from the query position, we saw that

- **1 head is not enough** for any choice of dimension, embeddings, positional embeddings, or linear readout.
- **2 heads are enough**, via the explicit construction above.

Together, these establish that two attention heads are necessary and sufficient to compute XOR with a logistic regression probe.

Open questions. A few natural directions this raises:

- **Parity on n bits:** How does the minimum number of heads scale with input length?
- **Wider implications:** *The geometric constraint is not specific to XOR or binary inputs: for any two token values $A \neq B$, a single attention head cannot linearly separate “same” inputs $\{(A, A), (B, B)\}$ from “mixed” inputs $\{(A, B), (B, A)\}$. This is a purely structural consequence of the weighted-average form of attention, independent of dimension or parameters. Where else does this diagonal-vs-off-diagonal bottleneck limit single-head expressivity?*